

# Power Consumption Analysis of Replicated Virtual Applications

Pietro Piazzolla<sup>1</sup>(✉), Gianfranco Ciardo<sup>2</sup>, and Andrew Miner<sup>2</sup>

<sup>1</sup> Department of Electronics, Information and Bioengineering,  
Politecnico di Milano, Milano, Italy  
`pietro.piazzolla@polimi.it`

<sup>2</sup> Department of Computer Science, Iowa State University, Ames, USA  
`{ciardo,asminer}@iastate.edu`

**Abstract.** The search for green IT has inspired a wide spectrum of techniques for power management. In a data center where computational power is provided by means of virtualised resources, like virtual machines, the policy to allocate them on physical servers can strongly impact the power consumption of the entire system. We propose a generalised stochastic Petri net model to investigate the contribution to energy efficiency due to different allocation and deallocation policies.

**Keywords:** Energy efficiency · Generalised stochastic Petri nets · Virtualised datacenters · Allocation policies · Performance evaluation

## 1 Introduction

Research to improve IT infrastructure sustainability has inspired a wide spectrum of techniques for power management that exploit, albeit in different ways, two types of basic mechanisms: the dynamic scaling of system components' performance (Dynamic Performance Scaling) and the dynamic hibernation of components (Dynamic Component Deactivation), see e.g.[1]. The main assumption underlying these techniques is that a system experiences a workload that varies over time, allowing for component adjustments. Among the techniques developed to improve energy efficiency in data centers, there are some that work on the system load, providing algorithms, heuristics or policies to schedule it among several servers. In a data center where computational power is provided to users by means of virtualised resources, for example virtual machines (VM), the policy followed to allocate them on physical hosts, or machines (PM), can strongly impact the power consumption of the whole system. In particular, by using different VM placement strategies, it is possible to control host utilization. For example, consolidating several VMs on a single PM allows some other physical machine to be in an idle state, thus lowering system energy consumption. In several works [3], the instantaneous power consumption is shown to have a linear

relation with CPU utilization, but even if the utilization is zero, the machine still consumes a large amount of energy just for being on and ready to process incoming requests. The power required by each idle server,  $P_{idle}$ , can heavily affect the consumption of the entire system. Strategies like consolidation can be used to reduce  $P_{idle}$  by sharing its contribution among all the VMs running on the same server. However, consolidation alone will only increase the number of idle machines: to be effective, it must be used in conjunction with a Dynamic Component Deactivation strategy, in order not to pay the power consumption  $P_{idle}$  for those hosts that are not utilised.

In this paper we are interested in studying how different VM allocation and release strategies impact energy consumption of a virtualised datacenter. Placement scheduling strategies, or *policies*, define from which available host to allocate the resources for a newly requested VM, while deallocation policies define from which server to release no longer required ones. We focus on systems running groups of identical VM instances, all replicating the services of the same application. Such an application is able to scale its number of instances according to the workload it handles. Since all the VMs are identical replicas and their allocation and deallocation depends only on the number of requests the application receives, it is possible to develop different strategies with different energy efficiency outcomes.

We propose a generalised stochastic Petri net (GSPN) model of a virtualised datacenter to measure the capability of each policy to reduce the number of PMs powered on, thus reducing  $P_{idle}$  consumption. This paper is one of the few to focus on the reduction of this power index, as well as one of the few to explicitly include deallocation policies in its analysis.

The rest of this paper is organized as follows. Section 2 gives a brief overview of related literature. Section 3 presents the main assumptions about the specific problem addressed in this paper and the analyzed policies. Section 4 presents the model of the virtualised datacenter in terms of its parameters and discusses the power consumption measures we adopt. The Petri net model is the focus of Section 5, while Section 6 presents results. Section 7 concludes the paper.

## 2 Related Work

A common problem in data center management is resource allocation in the presence of workloads having fluctuating intensities. In the literature, there are several works that deal with the optimal allocation of resources in virtual environments, aiming at different goals and exploiting different techniques. Of these works, many exploit probabilistic techniques and models [2, 14] to maximise selected performance indices.

Motivated by the need to understand and reduce energy waste and its related costs, in the last decade, researchers focused on devising techniques to optimise the power management of servers in large data centers. An exhaustive survey on this specific topic can be found in [1] and references therein. Early works on power consumption [11] propose policies to dynamically turn on or off cluster nodes,

according to the system workload, but without addressing the issues introduced by virtualization. Starting from [10], power management techniques have been explored in the context of virtualised systems. The problem of dynamic provisioning of VMs for multitier web applications according to the current workload (number of incoming requests), in an efficient resources management perspective, is the topic of [8]. In several works [12], average data center power consumption is optimised by means of diverse power management policies. Often the proposed solutions exploits consolidation and migration of VMs as opposed to this paper. In [13] migration is also not considered, as the objective is to improve the utilization of resources, leading to reduced energy consumption. Each application can be deployed using several VMs instantiated on different PMs. The resources are allocated to applications proportionally according to the applications' priorities. A similar approach, studying power consumption and replication of services, is presented in [9] and extended in [4].

### 3 Scenario

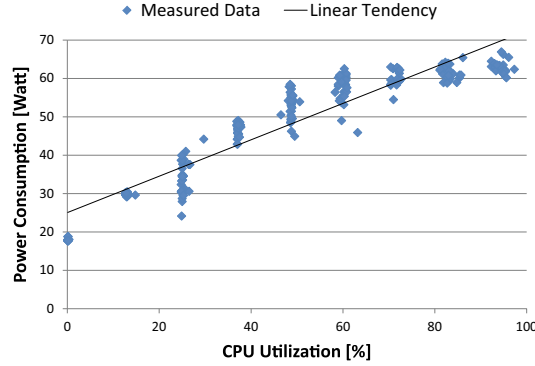
The computational and storage power of a datacenter is usually leveraged to execute *applications* whose services can be accessed by end users. We focus on those applications able to automatically scale their demand of system resources to satisfy a different workload of requests. If the resources of the system are provided in form of VMs, and the workload increases, then further VMs are allocated to that particular application. We consider that the VMs providing the application services can be replicated multiple times to serve a larger number of requests. These replicas are identical to one another, providing the same services and are used to scale the system. For the purpose of this work, the life time of a VM, i.e., the interval of time between its instantiation and its deallocation, is orders of magnitude higher than that of the single incoming job it serves. Once the application's request for further instances is acknowledged by the system, the new VM(s) instance(s) must be assigned to the available PM(s).

Different scheduling policies can be implemented to allocate the resources for new VMs among the available servers of the datacenter [15]. Some of the most common are:

- *Random*: a randomly determined PM is selected to host the new VM.
- *Round-robin*: the VM is placed on the next available PM in a sequence.
- *Least loaded server*: the VM is placed on the PM with the most available resources.
- *Most loaded server*: the VM is placed on the PM with the fewest available resources.

In the literature the allocation of VMs among the PMs is performed using optimization algorithms, but these solutions often require time and knowledge about the load of each application, a knowledge that is rarely available beforehand.

When the peak of requests terminates, the application releases all the excess VMs, scaling down its resources requirements. Different release policies can be



**Fig. 1.** Power consumption vs Utilization [3]

implemented to determine which VM is to be turned off. In this work we consider the following release policies: a randomly determined PM is selected to release the VM (*Random*), the VM is removed from the PM with the most available resources (*Least loaded server*), the VM is removed from the PM with the fewest available resources (*Most loaded server*).

The specific policy to allocate and release resources for VMs determines the utilization levels of the various hosts in a datacenter. In Fig. 1, from [3], the typical relation between the utilization of a server and its power consumption is shown. Although a less utilised server will consume less energy, as it can be seen, even if the utilization is zero, the machine still consumes energy simply because it is powered on. To reduce their  $P_{idle}$  contribution, unutilised servers must be put in a state in which their power consumption is negligible or null, using one or more Dynamic Component Deactivation techniques. Policies can be defined to determine when such servers are to be turned on or off. We assume that a server is turned off when it hosts no VMs, and is turned on when no resources are available in the pool of current active servers to allocate a new VM.

We want to study how the policies that determine the placement and release of virtual machines among PMs can influence the power consumption of a datacenter. In particular, allocating and deallocating resources for VM instances according to a specific strategy can determine different levels of utilization among the servers that may put a higher or lower number of them in idle state. Since idle PMs can be deactivated to save power, policies can be seen as a tool to increase power efficiency by reducing the  $P_{idle}$  contribution.

## 4 Model Description

Consider a system composed by a number  $M$  (large but finite) of homogeneous PMs, each able to allocate  $C$  resources for running VMs. These PMs can be either *powered on* or *powered off*. In the first case, they are able to serve requests while in the second case they are not, but will consume (little or) no power. We assume

the power consumption of a datacenter is proportional to the number of PMs powered on. We ignore startup or shutdown energy costs. The system is used by  $K$  different applications (basically classes of VMs). The services each application provides are based on a virtual server that can be replicated a maximum of  $N_k$  ( $1 \leq k \leq K$ ) times to allow heavier workloads to be served.  $N_k$  represents the degree of parallelization that application  $k$  can achieve. Once instantiated on a server  $m$ , a replica  $n$  cannot be moved to another server. That is, our model does not account for server migration. As [1] noticed, VM migration leads to time delays and performance overheads that require a dedicated analysis which is outside the scope of this paper. The state of the system is a collection of  $N_k$  variables per application  $k$ , and one per PM  $m$ . Let us call  $n_{k,i}$  the variable associated to the  $i$ -th VM that is hosting services for application  $k$  (with  $1 \leq i \leq N_k$ ). Its domain is  $n_{k,i} \in \{0, \dots, M\}$  indicating that the replica is not used (if  $n_{k,i} = 0$ ) or the number of the PM to which it is allocated ( $n_{k,i} = m$ ). The boolean variable  $\gamma_m$  indicates whether server  $m$  is powered off ( $\gamma_m = 0$ ) or not ( $\gamma_m = 1$ ). We assume that there are always resources available in the system to satisfy a request to allocate one more VM, that is:

$$M \cdot C \geq \sum_{k=1}^K N_k. \quad (1)$$

Two different types of events can change the state of the system:

- $\alpha$  events: request to allocate ( $\alpha_k^+$ ) or deallocate ( $\alpha_k^-$ ) a new VM. Allocating a new instance  $n$  for an application  $k$  is always possible unless  $n = N_k$ . Deallocating is always possible provided there is at least one  $n \neq 0$  for an application  $k$ .
- $\beta$  events: requests to power on ( $\beta^+$ ) or off ( $\beta^-$ ) a PM. Powering on a new PM occurs when no resources are available to allocate a new VM. On the converse, when a PM has no VMs it can be shut down. Powered off PMs do not contribute to system power consumption.

For each application  $k$  ( $1 \leq k \leq K$ ), the trigger of  $\alpha_k^+$  events follows a Poisson Process with an interarrival rate  $\lambda_k$ . Events  $\alpha_k^-$  for application  $k$  trigger following a Poisson Process with service rate  $\mu_k$ . Since we consider the lifetime of a VM to be orders of magnitude longer than that of the requests it serves, an  $\alpha_k^-$  event is fired when the number of requests to application  $k$  can be served by one fewer VM. The allocation policies listed in Section 3 define how the state of the system changes after the occurrence of an event  $\alpha$  or  $\beta$ . According to the focus of this study, the most significant parameter that we want to minimize is the  $P_{idle}$  required by a running PM. We define the energy consumption function or *efficiency*  $E(t)$  as the number of PMs powered off at time  $t$ . In particular, since we assume that no time is required to start or stop a PM and that  $\beta$  events are immediate, then new PMs are instantaneously available while PMs without VMs on it are automatically turned off. Let  $n_{VM}(m, t)$  be the number of VMs allocated on PM  $m$  at time  $t$ , and let us denote  $\mathbf{1}(\phi)$  the indicator function that

returns 1 if predicate  $\phi$  is true, and 0 otherwise. We can then define  $E(t)$  as:

$$E(t) = \sum_{m=1}^M \mathbf{1}(n_{VM}(m, t) = 0) \quad (2)$$

To compute the power consumption of the running PMs, instead, a more accurate approach is required. It has been shown [7] that a good approximation of the power consumption of a server can be described by a linear function of the utilization:

$$P(U) = P_{idle} + U \cdot (P_{max} - P_{idle}) \quad (3)$$

where  $P_{max}$  is the maximum power consumption that a given PM will have. We can approximate the utilization as  $U(n) = n/C$ , where  $n$  is the number of VMs running on the considered PM. To measure the energy consumption of the system we define an *estimated power consumption function*  $P(t)$  that considers the number of PMs powered on at time  $t$  and the number of VMs running on them. We can then define  $P(t)$  as:

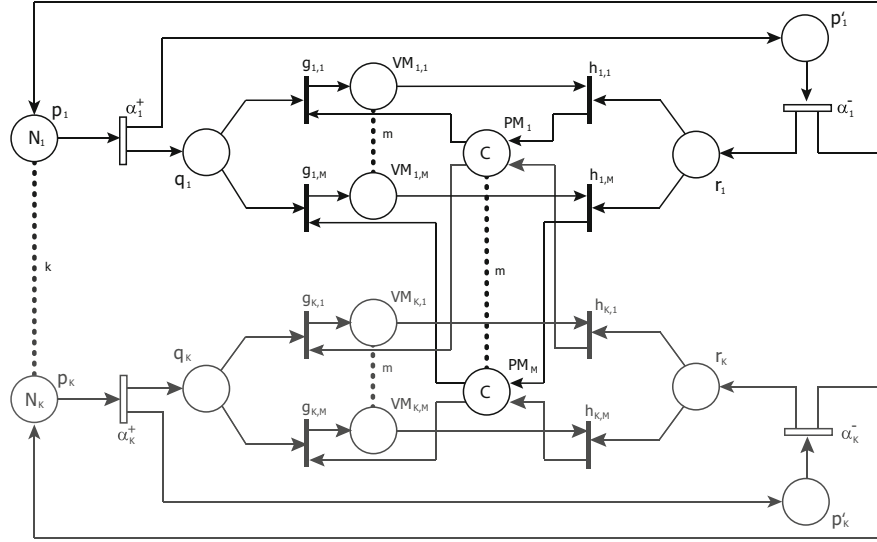
$$P(t) = \sum_{m=1}^M \left( \mathbf{1}(n_{VM}(m, t) \geq 1) \cdot P_{idle} + \frac{n_{VM}(m, t)}{C} \cdot (P_{max} - P_{idle}) \right) \quad (4)$$

If the  $P_{idle}$  consumption is higher than the power consumption per VM (that is, considering  $P_{max}$  and PM capacity  $C$ ), then Eq. 2 correctly accounts for this expenditure. Otherwise, if the power consumption per VM is higher than  $P_{idle}$  consumption, Eq. 4 provides a better approximation because it accounts for different PM types.

## 5 Petri Net

Fig. 2 shows the proposed GSPN model of the system. Places  $p_k$  represent virtual machines not (yet) requested for allocation by application  $k$  and may contain a number of tokens up to the maximum number of replicas allowed per application,  $N_k$ . Places  $p'_k$  are their complementary places ( $\#p'_k = N_k - \#p_k$ , where  $\#x$  signifies the number of tokens in place  $x$ ) and contain the number of running VMs for each application  $k$ .

The available PMs are represented by  $PM_m$  places, each holding up to  $C$  tokens. These tokens represent the resources each  $PM_m$  can provide to run VMs. For each applications  $k$ , places  $VM_{k,m}$  represent the number of running VMs an application has allocated on PM  $m$ . When transition  $\alpha_k^+$  fires for application  $k$ , the request for a new VM is issued (token in  $q_k$ ) and, according to the given policy (see Section 5.1), one of the immediate transitions  $g_{k,m}$  fires, placing the VM on one of the available PMs (adding one token in  $VM_{k,m}$ ). When requests arrive, immediate transitions  $g_{k,m}$  can fire only if there are resources available in the corresponding PM  $m$  (tokens in  $PM_m > 0$ ). When the  $\alpha_k^-$  transition is fired for application  $k$ , the request for stopping one of its running VMs is issued. This transition may fire only if there are running VMs for the application (tokens in  $p'_k > 0$ ). Once there are tokens in  $r_k$ , VMs and PMs resources can be released following a policy that enables one of the immediate transitions  $h_{k,m}$ .



**Fig. 2.** Petri net model of the system. The dotted lines indicate replications of subnets due to replication of places.

### 5.1 Modelling the Policies

**Random** placement and release policies introduced in Section 3 can be implemented with no changes to the model in Fig. 2, as the random selection of one of the available  $VM_{k,m}$  places naturally corresponds to PN semantics.

**Most-Least Loaded Server** placement policies can be implemented as a guard introduced to the immediate transitions  $g_{k,m}$ , while the corresponding release policies require a guard for  $h_{k,m}$  transitions. If we consider as the *least loaded server*  $PM_m$  the one with the highest capacity available we can define, for the model in Fig. 2, the following guards as the *Least loaded server* placement and release policies:

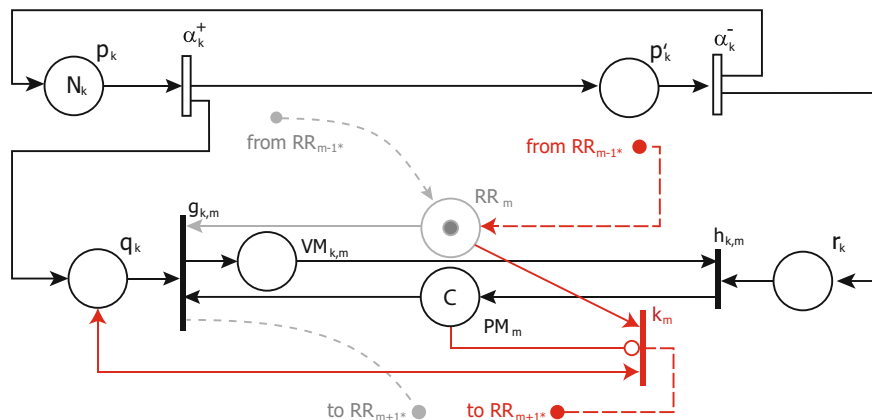
$$g_{k,m} \equiv h_{k,m} \equiv \bigwedge_{i \neq m} (\#PM_m \geq \#PM_i) \quad (5)$$

where the  $\wedge$  symbol is the logical *AND* among the propositions.

The *most loaded server* is the  $PM_m$  with the lowest capacity available, then for the model in Fig. 2, we can define the following guards as the *Most loaded server* placement and release policies:

$$g_{k,m} \equiv h_{k,m} \equiv \bigwedge_{i \neq m} (\#PM_m \leq \#PM_i) \quad (6)$$

**Round Robin Policy.** To be implemented as a placement policy requires the model in Fig. 2 to be changed. The model in Fig. 3 takes into account such changes and models the policy with the addition of a subnet whose places and



**Fig. 3.** Petri net of the model with round robin allocation policy

arcs are grey. The token in  $RR_m$  place represents the server  $m$  next in line to receive the allocation of a new VM. Once server  $m$  receives the VM, the round robin token is moved to the next server. The  $*$  character indicates that once the token is in  $RR_M$ , that is the last place of the round robin cycle, the next PM in line will be  $PM_1$ . In case a given  $PM_m$  runs out of resources, the subnet depicted in red make possible for the round robin token to skip that  $m$  and find the next available PM, thanks to the firing of  $k_m$  transitions. Bi-directional arcs  $(q_k, k_m)$  enable this skipping only when a new VM allocation is issued, thus avoiding the round robin token to loop forever when there are no free PMs. In this paper, we implement round robin as a placement policy only.

## 6 Experimental Results

This section presents results obtained solving the models of the previous sections using the SMART [5] tool (Stochastic Model checking Analyzer for Reliability and Timing). SMART takes in input a GSPN and generates the underlying continuous-time Markov chain (CTMC). To compute the steady-state distribution of the CTMC for models in Fig. 2 and 3, we use the Gauss-Seidel option (`#Solver GAUSS_SEIDEL`) with an exact symbolic representation of the transition rate matrix (`#SolutionType EXACT_EVMDD`).

We ran tests with different system configurations, each implementing a specific pair of “allocation–deallocation” policies, using the codes *Rnd* (random), *RR* (round robin), *LL* (least-loaded), *ML* (most-loaded). Some of these combinations correspond to real datacenters configurations, other are considered for comparative purposes. We used an i7 ASUS machine, running Ubuntu 14.04 OS.

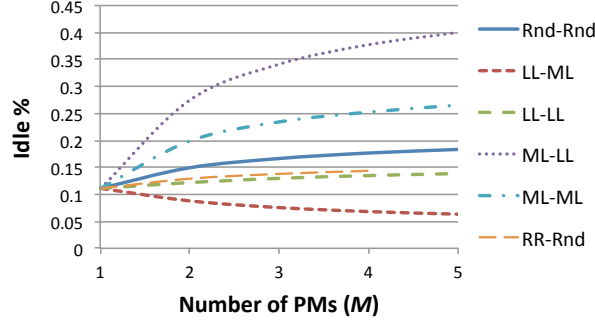


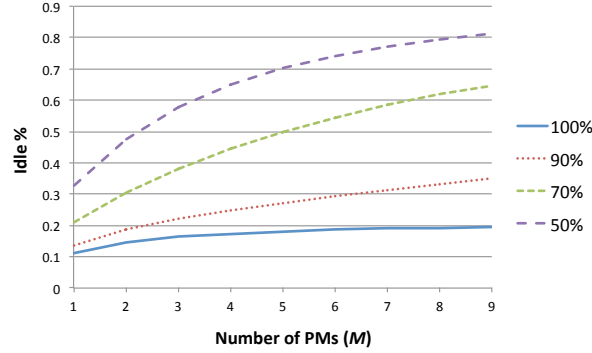
Fig. 4. Results for the first set of experiments

### 6.1 Two Application Classes

For the first set of experiments, consider a simple cluster with  $K = 2$  different classes of applications. All PMs are homogeneous and can host up to  $C = 4$  VMs. Each application class requests the allocation of single core VMs only. The interarrival time distribution of VM allocation requests (transitions  $\alpha_k^+$ ) is exponential with rate  $\lambda_1 = 1$  for class 1 and  $\lambda_2 = 3$  for class 2. Both rates are expressed as number of VM allocation requests per hour. While not particularly realistic, the values attributed to them allow to focus on the policies' contribution to system behavior. Deallocation (transitions  $\alpha_k^-$ ) rates per class are  $\mu_1 = \lambda_1$  and  $\mu_2 = \lambda_2$ , respectively, expressed as number of VM deallocation requests per hour. For each policy, we consider up to  $M = 5$  PMs. Since we want to test a fully utilized system,  $N_k$  is set according to Eq. 1 ( $N_k = 4 \cdot M/2$ ).

Fig. 4 shows results for these parametrizations. The  $x$ -axis corresponds to the number of PMs composing the system, the  $y$ -axis to the percentage of idle hosts, in steady-state. According to Eq. 2, a higher percentage of unused PMs implies a higher energy efficiency. From Fig. 4 it is possible to see that the percentage of unused PMs is affected by different policies. The  $ML-LL$  policy clearly outperforms the others as the total number of PMs increases, reaching nearly 40% idle machines in steady state. Even with different values, all policies show a higher percentage of idle hosts as  $M$  increases with the exception of  $LL-ML$ . Here, the allocation on new VMs on the least loaded (possibly empty) PM, together with a deallocation strategy that hardly unloads a given PM completely, contributes to an increase of power consumption as  $M$  increases.

Using an approximate solution (`#SolutionType APPROXIMATE_EVMDD`) for the  $Rnd-Rnd$  policy, we raise the number of PMs up to  $M = 9$  and use different *load factors*, that is, different values of  $\rho_k = \lambda_k / \mu_k$ . We only consider values of  $\rho \leq 1$  to test a system whose load intensity is not requiring its full capacity. In such a situation, the probability of having idle servers is sufficient to obtain benefits from the application of energy saving strategies. The service rates are fixed to  $\mu_1 = 1$  and  $\mu_2 = 3$ , while  $\lambda_1$  and  $\lambda_2$  are scaled to obtain different  $\rho$  values (for simplicity, we assume  $\rho_1 = \rho_2$ ).



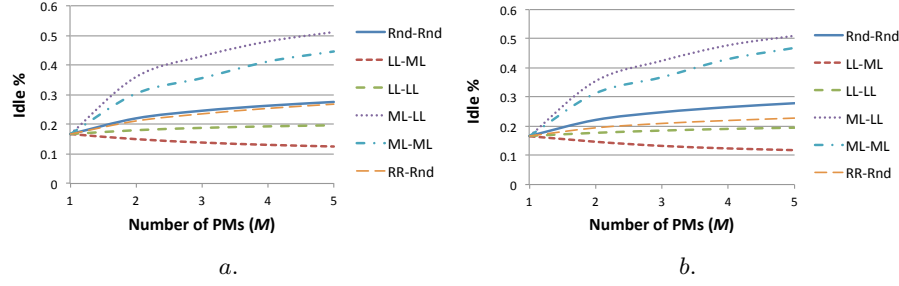
**Fig. 5.** Approximate results for the *Rnd-Rnd* policy under different load intensities

Fig. 5 shows the results. As expected, when  $\rho$  decreases, the probability to have idle machines increases. Unless noted otherwise, the rest of the paper assumes  $\lambda = \mu$  to highlight the policy effects on the power preservation.

## 6.2 Modelling Different Types of VMs

For the second set of experiments we improve the characterization of different application classes. In particular, we allow different resource demands for each application, in terms of the number  $C$  of resources required by their VMs, thus providing a more accurate representation of the investigated system. The models in Fig. 2 and 3 are modified by adding a weight  $w_k$  to the arcs connecting places  $PM_m$  to transitions  $g_{k,m}$ , representing this higher resources demand by class. The same weight  $w_k$  is applied to the arcs from transitions  $h_{k,m}$  to places  $PM_m$ , representing the deallocation of the same resource amount when a VM of that class is released. To satisfy the assumption of Eq. 1, the value of  $N$  for each class is now divided by the resources demands  $w_k$ , i.e.,  $N_k = M \cdot C / (K \cdot w_k)$ .

Fig. 6 shows the results for the improved models. For these tests, we use  $w_1 = 1$  and  $w_2 = 2$  and investigate a different load balance: in Fig. 6a,  $\lambda_1 = \mu_1 = 1$  while  $\lambda_2 = \mu_2 = 3$  (class 2 is the fastest but also the most demanding in terms of resources); in Fig. 6b,  $\lambda_1 = \mu_1 = 3$  while  $\lambda_2 = \mu_2 = 1$  (class 2 is the slowest but still the most demanding one). The comparison of these two figures reveals an interesting behavior. For some but not all the cases, the resulting percentage of idle machines is the same, suggesting that policies are not influenced in the same way by different load balancing, revealing some that are more resilient to workload characteristics. In particular, *ML-ML* results in slightly more idle servers when class 2 is the slowest but most resource demanding, while *RR-Rnd* shows the opposite behavior. Another interesting fact can be observed when the system has 3 PMs: on the *ML-ML* and *ML-LL* curves there is a sort of “step”. A possible reason is an uneven allocation of VMs that, for some states, prevents the allocation of a new 2-core VM on the most loaded server, requiring to allocate



**Fig. 6.** Results for the improved model. *a*: class 2 application is the fastest; *b*: class 2 application is the slowest. In both, class 2 is the most resource demanding.

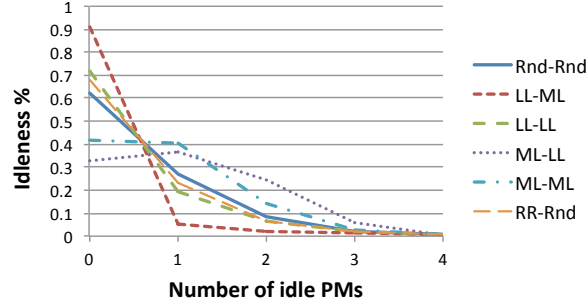
it on a (possibly) unused one, instead. This effect is presumably less evident as the number of PMs raises.

### 6.3 Performance Analysis and Energy Consumption

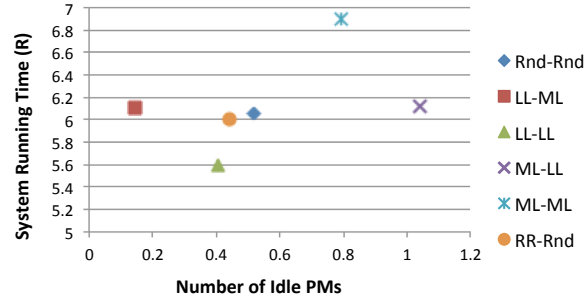
This section increases accuracy by including performance degradation of a PM when the number of running VMs on it increases. To this end, the firing of transition  $\alpha_k^-$  is slowed by a factor 10 for each resource used. For the tests in this section, we focus on a specific parametrization of the model: we fix the number of PMs to  $M = 4$ , all providing the same number  $C = 4$  of resources.  $K = 2$  applications run in the system, each with different allocation and deallocation rates ( $\lambda_1 = \mu_1 = 3$ ,  $\lambda_2 = \mu_2 = 1$ ) and different resource demands ( $w_1 = 1$ ,  $w_2 = 2$ ). The maximum number of instances per application is  $N_1 = 8$  and  $N_2 = 4$ , respectively. Fig. 7 shows, for each policy, the steady-state probability of having a given number of idle PMs. While the mean number of idle machines used in the previous sections is a useful estimate, we present here the distribution of this number per machine, to show more clearly how each policy affects energy consumption. For all cases in Fig. 7, the probability to have 3 or 4 unutilised PMs is negligible (less than 1%). Policies like *ML-LL* and *ML-ML* have the highest probability to leave idle 1 or 2 machines. Interestingly, *ML-LL* shows a lower probability to have no PMs idle than having only 1 in use.

The interference caused by multiple VMs active on the same PM can change the execution time of any batch process executed by that machine, especially when their combined workload pushes the utilization close to the maximum. Different allocation and deallocation policies can minimize or magnify this interference. We therefore introduce another performance measure for the model: the *system running time* ( $R$ ) representing the mean execution time of each allocated VM, considering their mutual interactions. The introduction of  $R$  helps relate energy consumption with system performance. This measure is equivalent to system response time for time-sharing systems and can be computed using the well known Little's Law ( $N = X \cdot R$ ).

Fig. 8 shows results for different policies, where  $R$  is expressed in hours. *ML-ML* shows a behavior opposite to *LL-LL*: *ML-ML* allows a system to have



**Fig. 7.** Steady state probability to have a given number of idle PMs, per policy



**Fig. 8.** Running time in hours computed using Little's Formula

more idle PMs, but at the cost of the worst system running time; *LL-LL*, while providing the best running time, requires more running PMs. Other policies provide similar  $R$ , but with very different energy consumption values.

In previous sections, we used Eq. 2 as a measure of energy consumption. To give a more accurate account of energy consumption per policy, we now apply Eq. 4. Fig. 9 shows results obtained by its use, for different values of  $P_{idle}$  and  $P_{busy}$ , as in [3]. The values used in Fig. 9a,  $P_{idle} = 18$  Watts and  $P_{busy} = 67$  Watts were measured on a laptop machine and may represent, in a virtualised datacenter, the newest servers on the market, with low consumption rates. In Fig. 9b,  $P_{idle} = 70$  Watts and  $P_{busy} = 160$  Watts values approximate instead measurements taken on a desktop i7 Asus machine. They may well represent the typical machine used as a server in a private cloud environment.

Different  $P_{idle}$  and  $P_{busy}$  values result in different power consumptions in the two figures, but they behave proportionally (the *LL-ML* always has the highest consumption). The only noticeable exceptions are *ML-LL* and *ML-ML*, as they tend to consolidate VMs over the fewest possible PMs. This strategy works best when  $P_{idle}$  is low, while it is less effective when it is high, as there is little gain in turning off a machine that does not consume much power when idle.

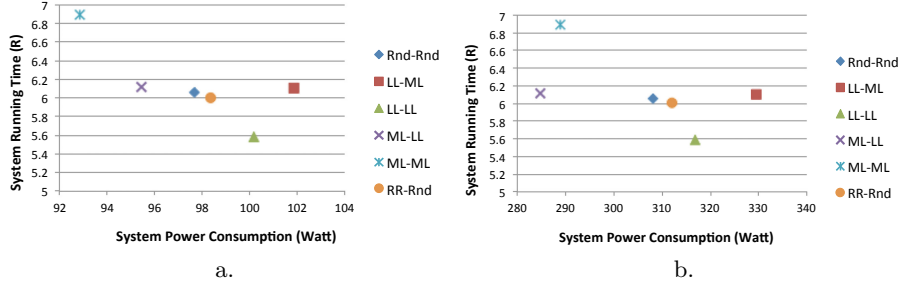
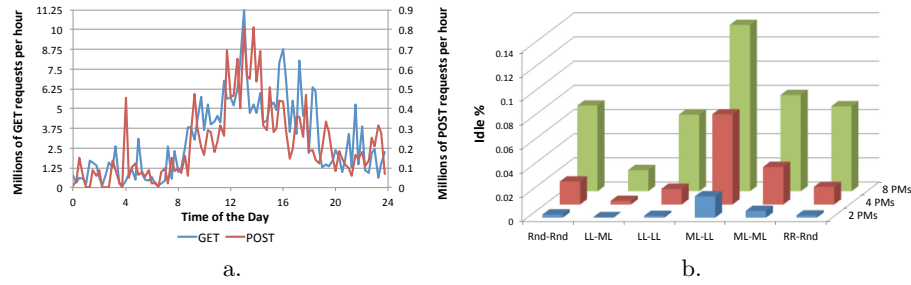


Fig. 9. Energy Consumption using different  $P_{idle}$  and  $P_{busy}$  values

#### 6.4 Using Real Traffic Data

As a final example, we present results obtained from parameterizing the model in Fig. 2 and 3 according to real internet traffic data. The traces, available at [6], are analyzed to determine  $\lambda_k$  and  $\mu_k$  rates for two classes of applications through a fitting procedure. In particular, after defining a time interval of 24 hours, we consider the workload generated by two classes of requests, *GET* and *POST*, during that interval in terms of number of operations issued every 15 minutes. In Fig. 10a, one can see the hourly distribution of requests. We assume that each virtual machine can handle a limited number of class operations per hour. During the day, as the workload fluctuates, the number of required VMs will vary. We set the maximum number of requests per hour handled by class 1 VMs at 1.25 million, while class 2 VMs can handle up to 100,000 requests per hour. In Fig. 10a, the main y-axis shows the thresholds for the first class of requests (*GET*) while the secondary y-axis shows the same for the second class (*PUT*). As it can also be seen from the figure's grid, we need a total of 18 VMs to handle the daily load, 9 for each class. From the traces it is possible to evaluate for how many 15 minute intervals a given number of VMs of a class is required to support the workload. We then set  $\lambda_k$  and  $\mu_k$  dependent to the marking of  $p_k$ , to reproduce, on average, the traces' throughput. Unlike for other tests, we assume that there is no degradation in performance when a PM utilization increases and that, in this case,  $\lambda_k(p_k) \neq \mu_k(p_k)$ . This requires to measure energy efficiency using Eq. 2, as the mean number of idle machines in steady state. Moreover, all the VMs requested by classes are of the same type ( $w_1 = w_2 = 1$ ). The maximum number of virtual machines required by one of the  $K = 2$  classes considered is fixed at  $N = 8$ . This is because we assume that at least 1 VM per class is always on to handle the minimum load expected and is thus excluded from the model. Fig. 10b presents results for three different combinations of  $M$  and  $C$  able to allocate the total of 16 VMs required. By Eq. 1,  $M \cdot C = 16$ ; thus, if the system has  $M = 2$ , then each PM machine has  $C = 8$  cores; if  $M = 4$ , then  $C = 4$ ; and, if  $M = 8$ , then  $C = 2$ .

The figure shows increasing probabilities to have idle PMs as  $M$  grows, as observed in Section 6.1. In this case, it is possible to compare how different



**Fig. 10.** a) Real traffic traces for two classes of requests. b) Comparing energy efficiency of different policies using real traffic data to set the model.

policies behave as PMs increase. Even if *ML-LL* provides the most idle PMs, policies like *LL-LL* and *Rnd-RR* seem to benefit more from a higher number of PMs, in particular showing, from  $M = 4$  to  $M = 8$ , an 380% idleness increase.

## 7 Conclusion

We used Petri net models to investigate how different VM allocation and deallocation strategies impact energy consumption in a virtualised datacenter. We showed that different performance objectives, in terms of response time versus power consumption can be achieved by choosing the appropriate placement policy. Future implementation will include features like times and power costs for server startup and shutdown. To extend the realism of the model, more complex policies as well as different classes of resources, like storage and network, will also be included in addition to the CPU. Due to state-space explosion, such model extensions will likely require discrete-event simulation for their study.

## References

1. Beloglazov, A., Buyya, R., Lee, Y.C., Zomaya, A., et al.: A taxonomy and survey of energy-efficient data centers and cloud computing systems. *Advances in Computers* **82**(2), 47–111 (2011)
2. Bannani, M.N., Menascé, D.A.: Resource allocation for autonomic data centers using analytic performance models. In: *Autonomic Computing, ICAC 2005*, pp. 229–240, June 2005
3. Cerotti, D., Gribaudo, M., Piazzolla, P., Pincioli, R., Serazzi, G.: Multi-class queuing networks models for energy optimization. In: *Proc. of 8th Int. Conf. Performance Evaluation Methodologies and Tools* (2014)
4. Cerotti, D., Gribaudo, M., Piazzolla, P., Serazzi, G.: Matching performance objectives for open and closed workloads by consolidation and replication. *Annals of Operations Research*, pp. 1–24 (2014)
5. Ciardo, G., Jones III, R.L., Miner, A.S., Siminiceanu, R.I.: Logic and stochastic modeling with SMART. *Performance Evaluation* **63**(6), 578–608 (2006). *Modelling Techniques and Tools for Computer Performance Evaluation*

6. Danzig, P., Mogul, J., Paxson, V., Schwartz, M.: The internet traffic archive. The archive is sited at the Lawrence Berkeley National Laboratory. <http://ita.ee.lbl.gov/html/traces.html>
7. Fan, X., Wolf-Dietrich Weber, W.-D., Barroso, L.A.: Power provisioning for a warehouse-sized computer. In: *Proce. of the 34th Int. Symposium on Computer Architecture*, pp. 13–23. ACM, New York (2007)
8. Gandhi, A., Harchol-Balter, M., Das, R., Lefurgy, C.: Optimal power allocation in server farms. In: *Proc. of the 11th Int. Conf. on Measurement and Modeling of Computer Systems*, pp. 157–168. ACM, NY (2009)
9. Gribaudo, M., Piazzolla, P., Serazzi, G.: Consolidation and replication of VMs matching performance objectives. In: Al-Begain, K., Fiems, D., Vincent, J.-M. (eds.) *ASMTA 2012. LNCS*, vol. 7314, pp. 106–120. Springer, Heidelberg (2012)
10. Nathuji, R., Schwan, K.: Virtualpower: Coordinated power management in virtualized enterprise systems. *SIGOPS Oper. Syst. Rev.* **41**(6), 265–278 (2007)
11. Pinheiro, E., Bianchini, R., Carrera, E.V., Heath, T.: Load balancing and unbalancing for power and performance in cluster-based systems (2001). <http://www2.ic.uff.br/julius/stre/pinheiro01load.pdf>
12. Raghavendra, R., Ranganathan, P., Talwar, V., et al.: No “power” struggles: Coordinated multi-level power management for the data center. *SIGARCH Comput. Archit. News* **36**(1), 48–59 (2008)
13. Song, Y., Wang, H., Li, Y., et al.: Multi-tiered on-demand resource scheduling for VM-based data center. In: *Proc. of the 9th Symposium on Cluster Computing and the Grid*, pp. 148–155. IEEE (2009)
14. Watson, B.J., Marwah, M., Gmach, D., et al.: Probabilistic performance modeling of virtualized resource allocation. In: *Proc. of the 7th int. Conference on Autonomic Computing*, pp. 99–108. ACM, NY (2010)
15. Xu, X., Hu, H., Hu, N., Ying, W.: Cloud task and virtual machine allocation strategy in cloud computing environment. In: Lei, J., Wang, F.L., Li, M., Luo, Y. (eds.) *NCIS 2012. CCIS*, vol. 345, pp. 113–120. Springer, Heidelberg (2012)